SQL Practical Demonstrations

A Comprehensive Guide with Queries, Outputs & Explanations

Database Management Systems

Practical Exercises 0 – 6

April 2026

Table of Contents

Practical 0: Database and Table Creation

> *Practical 0 focuses on creating a complete university database schema named 'prac0' with six interrelated tables: STUDENT, COURSE, ENROLLMENT, PROFESSOR, TEACHES, and SOCIETY. This practical demonstrates fundamental Data Definition Language (DDL) operations including database creation, table creation with appropriate data types, primary keys, foreign keys, check constraints, and composite keys. It also covers Data Manipulation Language (DML) operations with INSERT statements to populate all tables with sample data representing a small university ecosystem.*

Database and Table Setup

```
CREATE DATABASE IF NOT EXISTS prac0;
USE prac0;

DROP TABLE IF EXISTS SOCIETY;
DROP TABLE IF EXISTS TEACHES;
DROP TABLE IF EXISTS PROFESSOR;
DROP TABLE IF EXISTS ENROLLMENT;
DROP TABLE IF EXISTS COURSE;
DROP TABLE IF EXISTS STUDENT;
```

1. STUDENT Table

```
CREATE TABLE STUDENT(
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    Email VARCHAR(50),
    Department VARCHAR(30),
    Year INT,
    DOB DATE
);

INSERT INTO STUDENT VALUES (10001, 'Aryan', 'aryan@gmail.com', 'CS', 2, '2006-11-
05');
INSERT INTO STUDENT VALUES (10002, 'Shefali', 'shefali@gmail.com', 'BCOM', 2,
'2006-08-15');
INSERT INTO STUDENT VALUES (10003, 'Roy', 'roy@gmail.com', 'PS', 2, '2006-02-
14');
INSERT INTO STUDENT VALUES (10004, 'Bansal', 'bansal@gmail.com', 'BA', 2, '2006-
11-22');
```

Table 1 STUDENT Table Data

StudentID	Name	Email	Department	Year	DOB
10001	Aryan	aryan@gmail.com	CS	2	2006-11-05
10002	Shefali	shefali@gmail.com	BCOM	2	2006-08-15
10003	Roy	roy@gmail.com	PS	2	2006-02-14
10004	Bansal	bansal@gmail.com	BA	2	2006-11-22

2. COURSE Table

```
CREATE TABLE COURSE(
    CourseID INT PRIMARY KEY,
    Title VARCHAR(100),
    Credits INT,
    CourseType VARCHAR(20) CHECK (CourseType IN ('Fulltime', 'Parttime')),
    TotalSeats INT
);

INSERT INTO COURSE VALUES (101, 'CS', 24, 'Fulltime', 25);
INSERT INTO COURSE VALUES (102, 'BCOM', 24, 'Fulltime', 22);
INSERT INTO COURSE VALUES (103, 'PS', 24, 'Parttime', 45);
INSERT INTO COURSE VALUES (104, 'BA', 24, 'Parttime', 567);
```

Table 2 COURSE Table Data

CourseID	Title	Credits	CourseType	TotalSeats
101	CS	24	Fulltime	25
102	BCOM	24	Fulltime	22
103	PS	24	Parttime	45
104	BA	24	Parttime	567

3. ENROLLMENT Table

```
CREATE TABLE ENROLLMENT(
    EnrollID INT PRIMARY KEY,
    StudentID INT,
    CourseID INT,
    DateOfAdmission DATE,
    Grade CHAR(2),
    FOREIGN KEY(StudentID) REFERENCES STUDENT(StudentID),
    FOREIGN KEY(CourseID) REFERENCES COURSE(CourseID)
);

INSERT INTO ENROLLMENT VALUES (20001, 10001, 101, '2024-08-01', 'O');
INSERT INTO ENROLLMENT VALUES (20002, 10002, 102, '2024-08-01', 'A+');
INSERT INTO ENROLLMENT VALUES (20003, 10003, 103, '2024-08-01', 'A');
INSERT INTO ENROLLMENT VALUES (20004, 10004, 104, '2024-08-01', 'A-');
```

Table 3 ENROLLMENT Table Data

EnrollID	StudentID	CourseID	DateOfAdmission	Grade
20001	10001	101	2024-08-01	O
20002	10002	102	2024-08-01	A+
20003	10003	103	2024-08-01	A
20004	10004	104	2024-08-01	A-

4. PROFESSOR Table

```
CREATE TABLE PROFESSOR(
    ProfID INT PRIMARY KEY,
    Name VARCHAR(50),
    Department VARCHAR(30),
    Email VARCHAR(50)
);

INSERT INTO PROFESSOR VALUES (301, 'Dr. Sharma', 'CS', 'sharma@example.com');
INSERT INTO PROFESSOR VALUES (302, 'Dr. Verma', 'BCOM', 'verma@example.com');
INSERT INTO PROFESSOR VALUES (303, 'Dr. Gupta', 'PS', 'gupta@example.com');
INSERT INTO PROFESSOR VALUES (304, 'Dr. Singh', 'BA', 'singh@example.com');
```

Table 4 PROFESSOR Table Data

ProfID	Name	Department	Email
301	Dr. Sharma	CS	sharma@example.com
302	Dr. Verma	BCOM	verma@example.com
303	Dr. Gupta	PS	gupta@example.com
304	Dr. Singh	BA	singh@example.com

5. TEACHES Table

```
CREATE TABLE TEACHES(
    ProfID INT,
    CourseID INT,
    Semester VARCHAR(10),
    Year INT,
    PRIMARY KEY(ProfID, CourseID, Semester, Year),
    FOREIGN KEY(ProfID) REFERENCES PROFESSOR(ProfID),
    FOREIGN KEY(CourseID) REFERENCES COURSE(CourseID)
);

INSERT INTO TEACHES VALUES (301, 101, 'Fall', 2024);
INSERT INTO TEACHES VALUES (302, 102, 'Fall', 2024);
INSERT INTO TEACHES VALUES (303, 103, 'Spring', 2024);
INSERT INTO TEACHES VALUES (304, 104, 'Spring', 2024);
```

Table 5 TEACHES Table Data

ProfID	CourseID	Semester	Year
301	101	Fall	2024
302	102	Fall	2024
303	103	Spring	2024
304	104	Spring	2024

6. SOCIETY Table

```
CREATE TABLE SOCIETY(
    SocID INT PRIMARY KEY,
    SocName VARCHAR(50),
    FacultyInCharge VARCHAR(50)
);

INSERT INTO SOCIETY VALUES (401, 'Coding Club', 'Dr. Sharma');
INSERT INTO SOCIETY VALUES (402, 'Commerce Society', 'Dr. Verma');
INSERT INTO SOCIETY VALUES (403, 'Debate Society', 'Dr. Gupta');
INSERT INTO SOCIETY VALUES (404, 'Cultural Society', 'Dr. Singh');
```

Table 6 SOCIETY Table Data

SocID	SocName	FacultyInCharge
401	Coding Club	Dr. Sharma
402	Commerce Society	Dr. Verma
403	Debate Society	Dr. Gupta
404	Cultural Society	Dr. Singh

> **Note:** This practical establishes the foundation schema that subsequent practicals build upon. The tables demonstrate various constraint types including PRIMARY KEY, FOREIGN KEY, CHECK, and composite keys.

Practical 1: WHERE Clause Operators

> *Practical 1 demonstrates the use of various SQL comparison and logical operators within the WHERE clause to filter records from a single table. It covers equality conditions (=), range comparisons (>, <=, BETWEEN), pattern matching (LIKE), logical combinations (AND, OR, NOT), set membership (IN), and date comparisons. These operators form the foundation of data retrieval in SQL, allowing precise record selection based on business requirements.*

Table Setup

```
CREATE DATABASE IF NOT EXISTS prac1;
USE prac1;

DROP TABLE IF EXISTS employees;
CREATE TABLE employees(
    emp_id INT,
    first_name VARCHAR(30),
    last_name VARCHAR(30),
    department VARCHAR(20),
    salary INT,
    hire_date DATE
);

INSERT INTO employees VALUES (101, 'Rahul', 'Sharma', 'Sales', 45000, '2020-01-
15');
INSERT INTO employees VALUES (102, 'Priya', 'Mehta', 'HR', 55000, '2019-03-12');
INSERT INTO employees VALUES (103, 'Amit', 'Verma', 'IT', 60000, '2021-06-01');
INSERT INTO employees VALUES (104, 'Neha', 'Kapoor', 'Sales', 48000, '2022-07-
10');
INSERT INTO employees VALUES (105, 'Karan', 'Singh', 'Finance', 52000, '2020-11-
23');
```

Table 7 EMPLOYEES Table Data

emp_id	first_name	last_name	department	salary	hire_date
101	Rahul	Sharma	Sales	45000	2020-01-15
102	Priya	Mehta	HR	55000	2019-03-12
103	Amit	Verma	IT	60000	2021-06-01
104	Neha	Kapoor	Sales	48000	2022-07-10
105	Karan	Singh	Finance	52000	2020-11-23

Queries and Outputs

Query 1: Equality condition (=)

```
SELECT *
FROM employees
WHERE department = 'Sales';
```

Output:

emp_id	first_name	last_name	department	salary	hire_date
101	Rahul	Sharma	Sales	45000	2020-01-15
104	Neha	Kapoor	Sales	48000	2022-07-10

Query 2: Greater than (>)

```
SELECT first_name, salary
FROM employees
WHERE salary > 50000;
```

Output:

first_name	salary
Priya	55000
Amit	60000
Karan	52000

Query 3: Less than or equal to (<=)

```
SELECT first_name, department, salary
FROM employees
WHERE salary <= 48000;
```

Output:

first_name	department	salary
Rahul	Sales	45000
Neha	Sales	48000

Query 4: Range with BETWEEN

```
SELECT first_name, salary
FROM employees
WHERE salary BETWEEN 45000 AND 55000;
```

Output:

first_name	salary
Rahul	45000
Priya	55000
Neha	48000
Karan	52000

Query 5: Pattern matching with LIKE

```
SELECT first_name, last_name
FROM employees
WHERE first_name LIKE 'P%';
```

Output:

first_name	last_name
Priya	Mehta

Query 6: Logical AND

```
SELECT *
FROM employees
WHERE department = 'Sales' AND salary > 46000;
```

Output:

emp_id	first_name	last_name	department	salary	hire_date
104	Neha	Kapoor	Sales	48000	2022-07-10

Query 7: Logical OR

```
SELECT *
FROM employees
WHERE department = 'IT' OR department = 'Finance';
```

Output:

emp_id	first_name	last_name	department	salary	hire_date
103	Amit	Verma	IT	60000	2021-06-01
105	Karan	Singh	Finance	52000	2020-11-23

Query 8: NOT operator

```
SELECT first_name, department
FROM employees
WHERE NOT department = 'HR';
```

Output:

first_name	department
Rahul	Sales
Amit	IT
Neha	Sales
Karan	Finance

Query 9: Date condition

```
SELECT first_name, hire_date
FROM employees
WHERE hire_date > '2020-12-31';
```

Output:

first_name	hire_date
Amit	2021-06-01
Neha	2022-07-10

Query 10: IN operator

```
SELECT first_name, department
FROM employees
WHERE department IN ('Sales', 'Finance');
```

Output:

first_name	department
Rahul	Sales
Neha	Sales
Karan	Finance

Practical 2: Aggregate Functions

> *Practical 2 explores SQL aggregate functions which perform calculations on multiple rows and return a single value. It demonstrates COUNT(*) for row counting, AVG() for computing averages, MAX() and MIN() for finding extremes, and SUM() for totaling values. The practical also introduces the GROUP BY clause for grouping results by categories (such as department) and the HAVING clause for filtering grouped results. These functions are essential for data analysis and reporting tasks, enabling summarization of large datasets into meaningful metrics.*

Table Setup

This practical uses the `employees` table from Practical 1 (database `prac2`).

Table 8 EMPLOYEES Table Data

emp_id	first_name	last_name	department	salary	hire_date
101	Rahul	Sharma	Sales	45000	2020-01-15
102	Priya	Mehta	HR	55000	2019-03-12
103	Amit	Verma	IT	60000	2021-06-01
104	Neha	Kapoor	Sales	48000	2022-07-10
105	Karan	Singh	Finance	52000	2020-11-23

Queries and Outputs

Query 1: Count all employees

```
SELECT COUNT(*) AS total_employees
FROM employees;
```

Output:

total_employees
5

Query 2: Average salary

```
SELECT AVG(salary) AS avg_salary
FROM employees;
```

Output:

avg_salary
52000.0000

Query 3: Maximum salary

```
SELECT MAX(salary) AS highest_salary
FROM employees;
```

Output:

highest_salary
60000

Query 4: Minimum salary

```
SELECT MIN(salary) AS lowest_salary
FROM employees;
```

Output:

lowest_salary
45000

Query 5: Sum of all salaries

```
SELECT SUM(salary) AS total_salary
FROM employees;
```

Output:

total_salary
260000

Query 6: Count employees in Sales department

```
SELECT COUNT(*) AS sales_count
FROM employees
WHERE department = 'Sales';
```

Output:

sales_count
2

Query 7: Average salary by department

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

Output:

department	avg_salary
Finance	52000.0
HR	55000.0
IT	60000.0
Sales	46500.0

Query 8: Maximum salary by department

```
SELECT department, MAX(salary) AS max_salary
FROM employees
GROUP BY department;
```

Output:

department	max_salary
Finance	52000
HR	55000
IT	60000
Sales	48000

Query 9: Departments having more than 1 employee (HAVING)

```
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department
HAVING COUNT(*) > 1;
```

Output:

department	emp_count
Sales	2

Query 10: Total salary of employees hired after 2020

```
SELECT SUM(salary) AS total_salary
FROM employees
WHERE hire_date > '2020-12-31';
```

Output:

total_salary
108000

Practical 3: String and Date Functions

> *Practical 3 covers essential MySQL built-in functions for string manipulation and date/time operations. String functions include UPPER() and LOWER() for case conversion, LENGTH() for character counting, CONCAT() for string concatenation, LEFT() for substring extraction, and REPLACE() for text substitution. Date functions demonstrate YEAR() and MONTH() extraction, CURDATE() for current date retrieval, TIMESTAMPDIFF() for date difference calculations, DATE_ADD() for date arithmetic, and DATEDIFF() for day counting. These functions are crucial for formatting, transforming, and analyzing temporal and textual data in real-world applications.*

Table Setup

This practical uses the `employees` table from the `prac2` database.

Table 9 EMPLOYEES Table Data

emp_id	first_name	last_name	department	salary	hire_date
101	Rahul	Sharma	Sales	45000	2020-01-15
102	Priya	Mehta	HR	55000	2019-03-12
103	Amit	Verma	IT	60000	2021-06-01
104	Neha	Kapoor	Sales	48000	2022-07-10
105	Karan	Singh	Finance	52000	2020-11-23

Queries and Outputs

Query 1: Convert names to uppercase

```
SELECT UPPER(`first_name`) AS `FirstName_Upper`
FROM `employees`;
```

Output:

FirstName_Upper
RAHUL
PRIYA
AMIT
NEHA
KARAN

Query 2: Convert last names to lowercase

```
SELECT LOWER(`last_name`) AS `LastName_Lower`
FROM `employees`;
```

Output:

LastName_Lower
sharma
mehta
verma
kapoor
singh

Query 3: Find length of first names

```
SELECT `first_name`, LENGTH(`first_name`) AS `Name_Length`
FROM `employees`;
```

Output:

first_name	Name_Length
Rahul	5
Priya	5
Amit	4
Neha	4
Karan	5

Query 4: Concatenate first and last names

```
SELECT CONCAT(`first_name`, ' ', `last_name`) AS `Full_Name`
FROM `employees`;
```

Output:

Full_Name
Rahul Sharma
Priya Mehta
Amit Verma
Neha Kapoor
Karan Singh

Query 5: Extract first 3 letters of first name

```
SELECT LEFT(`first_name`, 3) AS `ShortName`
FROM `employees`;
```

Output:

ShortName
Rah
Pri
Ami
Neh
Kar

Query 6: Replace department name

```
SELECT REPLACE(`department`, 'Sales', 'Marketing') AS `Updated_Department`
FROM `employees`;
```

Output:

Updated_Department
Marketing
HR
IT
Marketing
Finance

Query 7: Extract year from hire date

```
SELECT `first_name`, YEAR(`hire_date`) AS `Hire_Year`
FROM `employees`;
```

Output:

first_name	Hire_Year
Rahul	2020
Priya	2019
Amit	2021
Neha	2022
Karan	2020

Query 8: Extract month from hire date

```
SELECT `first_name`, MONTH(`hire_date`) AS `Hire_Month`
FROM `employees`;
```

Output:

first_name	Hire_Month
Rahul	1
Priya	3
Amit	6
Neha	7
Karan	11

Query 9: Current date

```
SELECT CURDATE() AS `Today`;
```

Output:

Today
2026-04-29

Query 10: Calculate years of service

```
SELECT `first_name`, TIMESTAMPDIFF(YEAR, `hire_date`, CURDATE()) AS `Years_of_Service`
FROM `employees`;
```

Output:

first_name	Years_of_Service
Rahul	6
Priya	7
Amit	4
Neha	3
Karan	5

Query 11: Add 6 months to hire date (probation end)

```
SELECT `first_name`, DATE_ADD(`hire_date`, INTERVAL 6 MONTH) AS `Probation_End`
FROM `employees`;
```

Output:

first_name	Probation_End
Rahul	2020-07-15
Priya	2019-09-12
Amit	2021-12-01
Neha	2023-01-10
Karan	2021-05-23

Query 12: Difference in days since hire date

```
SELECT `first_name`, DATEDIFF(CURDATE(), `hire_date`) AS `Days_Since_Hire`
FROM `employees`;
```

Output:

first_name	Days_Since_Hire
Rahul	2296
Priya	2605
Amit	1793
Neha	1389
Karan	1983

Practical 4: GROUP BY and HAVING

> *Practical 4 provides an in-depth exploration of the GROUP BY and HAVING clauses for data aggregation and filtering. It begins with basic GROUP BY operations computing averages per department, then advances to multi-column grouping, and finally introduces HAVING for filtering aggregated results. The practical demonstrates counting employees, computing averages, maximums, minimums, and sums by department. It also covers grouping by derived values (YEAR of hire_date), combining WHERE with GROUP BY, and using multiple aggregate functions in a single query. These techniques are fundamental for producing summary reports and analytical dashboards.*

Table Setup

```
CREATE DATABASE IF NOT EXISTS prac4;
USE prac4;

DROP TABLE IF EXISTS employees;
CREATE TABLE employees(
    emp_id INT,
    first_name VARCHAR(30),
    department VARCHAR(20),
    salary INT
);

INSERT INTO employees VALUES (101, 'Rahul', 'Sales', 45000);
INSERT INTO employees VALUES (102, 'Priya', 'HR', 55000);
INSERT INTO employees VALUES (103, 'Amit', 'IT', 60000);
INSERT INTO employees VALUES (104, 'Neha', 'Sales', 48000);
INSERT INTO employees VALUES (105, 'Karan', 'Finance', 52000);

CREATE TABLE employee_1(
    emp_id INT,
    first_name VARCHAR(30),
    last_name VARCHAR(30),
    department VARCHAR(20),
    salary INT,
    hire_date DATE
);

INSERT INTO employees_1 VALUES (101, 'Rahul', 'Sharma', 'Sales', 45000, '2020-01-
15');
INSERT INTO employees_1 VALUES (102, 'Priya', 'Mehta', 'HR', 55000, '2019-03-
12');
INSERT INTO employees_1 VALUES (103, 'Amit', 'Verma', 'IT', 60000, '2021-06-01');
INSERT INTO employees_1 VALUES (104, 'Neha', 'Kapoor', 'Sales', 48000, '2022-07-
10');
INSERT INTO employees_1 VALUES (105, 'Karan', 'Singh', 'Finance', 52000, '2020-
11-23');
INSERT INTO employees_1 VALUES (106, 'Anjali', 'Rao', 'IT', 62000, '2021-02-20');
INSERT INTO employees_1 VALUES (107, 'Rohan', 'Das', 'HR', 53000, '2018-08-25');
```

Table 10 EMPLOYEES Table Data

emp_id	first_name	department	salary
101	Rahul	Sales	45000
102	Priya	HR	55000
103	Amit	IT	60000
104	Neha	Sales	48000
105	Karan	Finance	52000

Table 11 EMPLOYEES_1 Table Data

emp_id	first_name	last_name	department	salary	hire_date
101	Rahul	Sharma	Sales	45000	2020-01-15
102	Priya	Mehta	HR	55000	2019-03-12
103	Amit	Verma	IT	60000	2021-06-01
104	Neha	Kapoor	Sales	48000	2022-07-10
105	Karan	Singh	Finance	52000	2020-11-23
106	Anjali	Rao	IT	62000	2021-02-20
107	Rohan	Das	HR	53000	2018-08-25

Queries and Outputs

Query 1: Average salary by department

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

Output:

department	avg_salary
Finance	52000.0
HR	55000.0
IT	60000.0
Sales	46500.0

Query 2: Group by multiple columns (department and hire year)

```
SELECT department, hire_year, COUNT(*) AS emp_count
FROM employees
GROUP BY department, hire_year;
```

Output:

department	hire_year	emp_count
Finance	2020	1
HR	2018	1
HR	2019	1
IT	2021	2
Sales	2020	1
Sales	2022	1

Query 3: GROUP BY with HAVING (departments with more than 1 employee)

```
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department
HAVING COUNT(*) > 1;
```

Output:

department	emp_count
Sales	2

Query 4: Count employees in each department

```
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department;
```

Output:

department	emp_count
Finance	1
HR	1
IT	1
Sales	2

Query 5: Average salary by department

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

Output:

department	avg_salary
Finance	52000.0
HR	55000.0
IT	60000.0
Sales	46500.0

Query 6: Maximum salary in each department

```
SELECT department, MAX(salary) AS max_salary
FROM employees
GROUP BY department;
```

Output:

department	max_salary
Finance	52000
HR	55000
IT	60000
Sales	48000

Query 7: Minimum salary in each department

```
SELECT department, MIN(salary) AS min_salary
FROM employees
GROUP BY department;
```

Output:

department	min_salary
Finance	52000
HR	55000
IT	60000
Sales	45000

Query 8: Total salary by department

```
SELECT department, SUM(salary) AS total_salary
FROM employees
GROUP BY department;
```

Output:

department	total_salary
Finance	52000
HR	55000
IT	60000
Sales	93000

Query 9: Departments with more than 1 employee

```
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department
HAVING COUNT(*) > 1;
```

Output:

department	emp_count
Sales	2

Query 10: Count employees by year of hire

```
SELECT YEAR(hire_date) AS hire_year, COUNT(*) AS emp_count
FROM employees
GROUP BY hire_year;
```

Output:

hire_year	emp_count
2018	1
2019	1
2020	2
2021	2
2022	1

Query 11: Average salary by department for employees hired after 2020

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
WHERE hire_date > '2020-12-31'
GROUP BY department;
```

Output:

department	avg_salary
IT	61000.0
Sales	48000.0

Query 12: Departments and years having more than 1 employee

```
SELECT department, YEAR(hire_date) AS hire_year, COUNT(*) AS emp_count
FROM employees
GROUP BY department, YEAR(hire_date)
HAVING COUNT(*) > 1;
```

Output:

department	hire_year	emp_count
IT	2021	2

Query 13: Total salary and average salary by department

```
SELECT department, SUM(salary) AS total_salary, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

Output:

department	total_salary	avg_salary
Finance	52000	52000.0
HR	55000	55000.0
IT	60000	60000.0
Sales	93000	46500.0

Practical 5: LIKE and Wildcards

> *Practical 5 demonstrates pattern matching using the LIKE operator and wildcard characters in SQL. The percent (%) wildcard matches zero or more characters, while the underscore (_) wildcard matches exactly one character. This practical covers queries for finding names starting with specific letters, ending with particular characters, containing substrings, having characters at specific positions, and their negations using NOT LIKE. Additional queries explore practical patterns such as filtering by last name initials, excluding departments, and combining multiple pattern conditions.*

Table Setup

```
CREATE DATABASE IF NOT EXISTS prac5;
USE prac5;

DROP TABLE IF EXISTS Employees;
CREATE TABLE Employees(
    EmpID INT,
    EmpName VARCHAR(50),
    Department VARCHAR(20),
    City VARCHAR(30)
);

INSERT INTO Employees VALUES(1, 'Amit Sharma', 'HR', 'Delhi');
INSERT INTO Employees VALUES(2, 'Ravi Kumar', 'IT', 'Mumbai');
INSERT INTO Employees VALUES(3, 'Anita Verma', 'Finance', 'Pune');
INSERT INTO Employees VALUES(4, 'Reena Kapoor', 'IT', 'Delhi');
INSERT INTO Employees VALUES(5, 'Aman Gupta', 'Admin', 'Chennai');
INSERT INTO Employees VALUES(6, 'Rohit Singh', 'Finance', 'Kolkata');
```

Table 12 Employees Table Data

EmpID	EmpName	Department	City
1	Amit Sharma	HR	Delhi
2	Ravi Kumar	IT	Mumbai
3	Anita Verma	Finance	Pune
4	Reena Kapoor	IT	Delhi
5	Aman Gupta	Admin	Chennai
6	Rohit Singh	Finance	Kolkata

Queries and Outputs

Core LIKE Queries

Query 1: Display employees whose name starts with 'A'

```
SELECT * FROM Employees WHERE EmpName LIKE 'A%';
```

Output:

EmpID	EmpName	Department	City
1	Amit Sharma	HR	Delhi
3	Anita Verma	Finance	Pune
5	Aman Gupta	Admin	Chennai

Query 2: Display employees whose city ends with 'i'

```
SELECT * FROM Employees WHERE City LIKE '%i';
```

Output:

EmpID	EmpName	Department	City
1	Amit Sharma	HR	Delhi
2	Ravi Kumar	IT	Mumbai
4	Reena Kapoor	IT	Delhi
5	Aman Gupta	Admin	Chennai

Query 3: Display employees whose department contains the letter 'i'

```
SELECT * FROM Employees WHERE Department LIKE '%i%';
```

Output:

EmpID	EmpName	Department	City
2	Ravi Kumar	IT	Mumbai
3	Anita Verma	Finance	Pune
4	Reena Kapoor	IT	Delhi
5	Aman Gupta	Admin	Chennai
6	Rohit Singh	Finance	Kolkata

Query 4: Display employees whose name has 'a' as the second character

```
SELECT * FROM Employees WHERE EmpName LIKE '_a%';
```

Output:

EmpID	EmpName	Department	City
2	Ravi Kumar	IT	Mumbai

Query 5: Display employees whose city starts with any character and then 'e'

```
SELECT * FROM Employees WHERE City LIKE '_e%';
```

Output:

EmpID	EmpName	Department	City
1	Amit Sharma	HR	Delhi
4	Reena Kapoor	IT	Delhi

Query 6: Display employees whose name does not contain the letter 'a'

```
SELECT * FROM Employees WHERE EmpName NOT LIKE '%a%';
```

Output:

EmpID	EmpName	Department	City
6	Rohit Singh	Finance	Kolkata

Additional LIKE Queries

Query 7: Display employees whose name ends with the letter 'a'

```
SELECT * FROM Employees WHERE EmpName LIKE '%a';
```

Output:

EmpID	EmpName	Department	City
1	Amit Sharma	HR	Delhi
3	Anita Verma	Finance	Pune
5	Aman Gupta	Admin	Chennai

Query 8: Display employees whose last name starts with 'K'

```
SELECT * FROM Employees WHERE EmpName LIKE '% K%';
```

Output:

EmpID	EmpName	Department	City
2	Ravi Kumar	IT	Mumbai
4	Reena Kapoor	IT	Delhi

Query 9: Display employees whose City contains the letter 'n'

```
SELECT * FROM Employees WHERE City LIKE '%n%';
```

Output:

EmpID	EmpName	Department	City
3	Anita Verma	Finance	Pune
5	Aman Gupta	Admin	Chennai

Query 10: Display employees whose department does NOT start with 'F'

```
SELECT * FROM Employees WHERE Department NOT LIKE 'F%';
```

Output:

EmpID	EmpName	Department	City
1	Amit Sharma	HR	Delhi
2	Ravi Kumar	IT	Mumbai
4	Reena Kapoor	IT	Delhi
5	Aman Gupta	Admin	Chennai

Query 11: Display employees whose name has exactly 'ee' as the second and third characters

```
SELECT * FROM Employees WHERE EmpName LIKE '_ee%';
```

Output:

EmpID	EmpName	Department	City
4	Reena Kapoor	IT	Delhi

Pattern Variations

Query 12: Names ending with a specific letter ('i' in first name)

```
SELECT * FROM Employees WHERE EmpName LIKE '%i %';
```

Output:

EmpID	EmpName	Department	City
No records found			

Query 13: Names starting with a specific letter ('R')

```
SELECT * FROM Employees WHERE EmpName LIKE 'R%';
```

Output:

EmpID	EmpName	Department	City
2	Ravi Kumar	IT	Mumbai
4	Reena Kapoor	IT	Delhi
6	Rohit Singh	Finance	Kolkata

Query 14: Names containing certain letters ('m' anywhere in the name)

```
SELECT * FROM Employees WHERE EmpName LIKE '%m%';
```

Output:

EmpID	EmpName	Department	City
1	Amit Sharma	HR	Delhi
2	Ravi Kumar	IT	Mumbai
3	Anita Verma	Finance	Pune
5	Aman Gupta	Admin	Chennai

Query 15: Names where the first name ends with 'a'

```
SELECT * FROM Employees WHERE EmpName LIKE '%a %';
```

Output:

EmpID	EmpName	Department	City
No records found			

Query 16: Names starting with 'A' and ending with 'a'

```
SELECT * FROM Employees WHERE EmpName LIKE 'A%a';
```

Output:

EmpID	EmpName	Department	City
1	Amit Sharma	HR	Delhi
3	Anita Verma	Finance	Pune
5	Aman Gupta	Admin	Chennai

NOT LIKE Queries

Query 17: Exclude employees whose name starts with the letter 'A'

```
SELECT * FROM Employees WHERE EmpName NOT LIKE 'A%';
```

Output:

EmpID	EmpName	Department	City
2	Ravi Kumar	IT	Mumbai
4	Reena Kapoor	IT	Delhi
6	Rohit Singh	Finance	Kolkata

Query 18: Exclude employees whose city ends with 'i'

```
SELECT * FROM Employees WHERE City NOT LIKE '%i';
```

Output:

EmpID	EmpName	Department	City
3	Anita Verma	Finance	Pune
6	Rohit Singh	Finance	Kolkata

Query 19: Exclude employees whose department contains the letters 'in'

```
SELECT * FROM Employees WHERE Department NOT LIKE '%in%';
```

Output:

EmpID	EmpName	Department	City
1	Amit Sharma	HR	Delhi
2	Ravi Kumar	IT	Mumbai
4	Reena Kapoor	IT	Delhi

Query 20: Exclude employees whose name has 'm' as the second character

```
SELECT * FROM Employees WHERE EmpName NOT LIKE '_m%';
```

Output:

EmpID	EmpName	Department	City
2	Ravi Kumar	IT	Mumbai
3	Anita Verma	Finance	Pune
4	Reena Kapoor	IT	Delhi
6	Rohit Singh	Finance	Kolkata

Query 21: Exclude employees whose city contains the letter 'e'

```
SELECT * FROM Employees WHERE City NOT LIKE '%e%';
```

Output:

EmpID	EmpName	Department	City
2	Ravi Kumar	IT	Mumbai
6	Rohit Singh	Finance	Kolkata

Practical 6: INNER JOIN

> *Practical 6 explores INNER JOIN operations to combine rows from two or more tables based on related columns. It demonstrates equi-joins between Employees and Departments tables using the DeptID foreign key relationship. The practical covers basic join syntax, selecting specific columns from joined tables, applying WHERE clauses to filter joined results, sorting joined output with ORDER BY, and combining JOIN with aggregate functions and GROUP BY. These operations are essential for working with normalized relational databases where data is distributed across multiple tables.*

Table Setup

```
CREATE DATABASE IF NOT EXISTS prac6;
USE prac6;

DROP TABLE IF EXISTS Employees;
CREATE TABLE Employees(
    EmpID INT,
    EmpName VARCHAR(50),
    DeptID VARCHAR(10)
);

DROP TABLE IF EXISTS Departments;
CREATE TABLE Departments(
    DeptID VARCHAR(10),
    DeptName VARCHAR(30),
    Location VARCHAR(30)
);

INSERT INTO Departments VALUES('D001', 'IT', 'Delhi');
INSERT INTO Departments VALUES('D002', 'HR', 'Mumbai');
INSERT INTO Departments VALUES('D003', 'Finance', 'Delhi');
INSERT INTO Departments VALUES('D004', 'Admin', 'Bangalore');

INSERT INTO Employees VALUES(1, 'Amit Sharma', 'D001');
INSERT INTO Employees VALUES(2, 'Ravi Kumar', 'D002');
INSERT INTO Employees VALUES(3, 'Anita Verma', 'D001');
INSERT INTO Employees VALUES(4, 'Reena Kapoor', 'D003');
INSERT INTO Employees VALUES(5, 'Aman Gupta', 'D001');
INSERT INTO Employees VALUES(6, 'Rohit Singh', 'D004');
```

Table 13 Departments Table Data

DeptID	DeptName	Location
D001	IT	Delhi
D002	HR	Mumbai
D003	Finance	Delhi
D004	Admin	Bangalore

Table 14 Employees Table Data

EmpID	EmpName	DeptID
1	Amit Sharma	D001
2	Ravi Kumar	D002
3	Anita Verma	D001
4	Reena Kapoor	D003
5	Aman Gupta	D001
6	Rohit Singh	D004

Queries and Outputs

Query 1: Display employee names along with their department names

```
SELECT Employees.EmpName, Departments.DeptName
FROM Employees
INNER JOIN Departments
ON Employees.DeptID = Departments.DeptID;
```

Output:

EmpName	DeptName
Amit Sharma	IT
Ravi Kumar	HR
Anita Verma	IT
Reena Kapoor	Finance
Aman Gupta	IT
Rohit Singh	Admin

Query 2: Display employee names, department names, and locations

```
SELECT EmpName, DeptName, Location
FROM Employees INNER JOIN Departments
ON Employees.DeptID = Departments.DeptID;
```

Output:

EmpName	DeptName	Location
Amit Sharma	IT	Delhi
Ravi Kumar	HR	Mumbai
Anita Verma	IT	Delhi
Reena Kapoor	Finance	Delhi
Aman Gupta	IT	Delhi
Rohit Singh	Admin	Bangalore

Query 3: Display names of employees working in the 'IT' department

```
SELECT EmpName
FROM Employees INNER JOIN Departments
ON Employees.DeptID = Departments.DeptID
WHERE Departments.DeptName = 'IT';
```

Output:

EmpName
Amit Sharma
Anita Verma
Aman Gupta

Query 4: Display names of employees and their department locations in 'Delhi'

```
SELECT EmpName, Location
FROM Employees INNER JOIN Departments
ON Employees.DeptID = Departments.DeptID
WHERE Departments.Location = 'Delhi';
```

Output:

EmpName	Location
Amit Sharma	Delhi
Anita Verma	Delhi
Reena Kapoor	Delhi
Aman Gupta	Delhi

Query 5: List employees and their department details for only matching DeptIDs

```
SELECT *
FROM Employees INNER JOIN Departments
ON Employees.DeptID = Departments.DeptID;
```

Output:

EmpID	EmpName	DeptID	DeptName	Location
1	Amit Sharma	D001	IT	Delhi
2	Ravi Kumar	D002	HR	Mumbai
3	Anita Verma	D001	IT	Delhi
4	Reena Kapoor	D003	Finance	Delhi
5	Aman Gupta	D001	IT	Delhi
6	Rohit Singh	D004	Admin	Bangalore

Query 6: Display employee ID, name, department, and location ordered by department

```
SELECT Employees.EmpID, Employees.EmpName, Departments.DeptName,
Departments.Location
FROM Employees INNER JOIN Departments
ON Employees.DeptID = Departments.DeptID
ORDER BY Departments.DeptName;
```

Output:

EmpID	EmpName	DeptName	Location
6	Rohit Singh	Admin	Bangalore
4	Reena Kapoor	Finance	Delhi
2	Ravi Kumar	HR	Mumbai
1	Amit Sharma	IT	Delhi
3	Anita Verma	IT	Delhi
5	Aman Gupta	IT	Delhi

Query 7: Display employees from Delhi with their ID, name, and department

```
SELECT Employees.EmpID, Employees.EmpName, Departments.DeptName,
Departments.Location
FROM Employees INNER JOIN Departments
ON Employees.DeptID = Departments.DeptID
WHERE Departments.Location = 'Delhi';
```

Output:

EmpID	EmpName	DeptName	Location
1	Amit Sharma	IT	Delhi
3	Anita Verma	IT	Delhi
4	Reena Kapoor	Finance	Delhi
5	Aman Gupta	IT	Delhi

Query 8: Display count of employees per department with department name and location

```
SELECT Departments.DeptName, Departments.Location, COUNT(Employees.EmpID) AS
EmployeeCount
FROM Employees INNER JOIN Departments
ON Employees.DeptID = Departments.DeptID
GROUP BY Departments.DeptID, Departments.DeptName, Departments.Location;
```

Output:

DeptName	Location	EmployeeCount
IT	Delhi	3
HR	Mumbai	1
Finance	Delhi	1
Admin	Bangalore	1